Real-Time Financial Fraud Detection Using Supervised Machine Learning (Credit Card Fraud Detection)

Technical assessment report, with a small question-answering prototype

Navadeep Reddy | Data Scientist Technical Assessment (Topic 1)

284,807	492	4	SVM	0.546
transactions	fraud cases (0.173%)	models compared	final model	best macro-F1

At a glance. I trained four classifiers to spot fraudulent card transactions in a highly imbalanced public dataset, using a split that keeps the test set untouched. All four catch most fraud, but they differ a lot in false alarms; SVM gave the best balance, with Logistic Regression a close second. A small TF-IDF question-answering tool completes the brief, and the report is open about what is not finished yet.

1. Problem understanding and solution design

Banks lose money to card fraud, and customers lose trust when a genuine purchase gets blocked. The people who deal with this are fraud analysts, who review flagged transactions, and the customers whose cards are affected. The difficulty is that fraud is extremely rare: in the dataset I used, only 492 of 284,807 transactions (about 0.173%) are fraudulent. A model that labels everything as genuine would score 99.8% accuracy and catch no fraud at all, so accuracy alone can't be trusted here.

Input and output. The input is one transaction described by 30 numeric features (Time, Amount and 28 anonymised PCA components, V1 to V28). The output is a label, genuine (0) or fraud (1). I also built a small retrieval tool that answers plain-English questions about the project from a short text knowledge base.

Constraints and assumptions. The data is anonymised, so I can't interpret V1 to V28 in business terms. I assumed the labels are correct and that this one public snapshot is representative enough for a learning project. I treat missing a fraud as costlier than reviewing an extra genuine transaction, so I paid most attention to fraud recall, precision, PR-AUC and macro-F1, not accuracy.

Design decisions. I split the data first and only then balanced the training portion, because balancing before the split lets information leak into the test set and makes results look better than they are. I compared four very different models (a tree, a distance-based model, a linear model and a kernel model) so I could see how much the choice of algorithm matters on the same data.

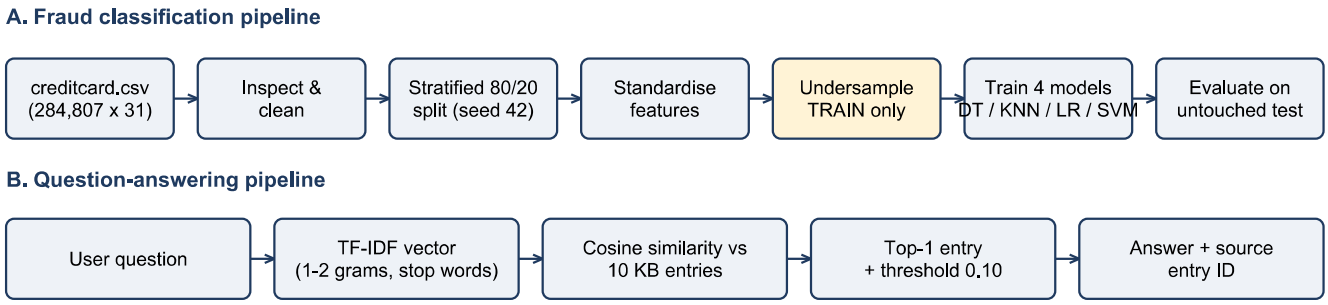


Figure 1. System architecture. The highlighted box marks the only place resampling happens.

2. Data collection and quality check

I used the public *Credit Card Fraud Detection* dataset on Kaggle (ULB Machine Learning Group): https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud. It is released under the Open Database License, Database Contents License: https://opendatacommons.org/licenses/dbcl/1-0/. Download date: 30 October 2026.

Column	Meaning
Time	Seconds elapsed since the first transaction in the data (0 to 172,792, roughly two days)
V1 to V28	Anonymised numeric components produced by PCA, which hides the original card details
Amount	Transaction amount (0 to 25,691.16)
Class	Target label: 0 = genuine, 1 = fraud

Quality check. The file has 284,807 rows and 31 columns (30 float64 features and one int64 label). isnull().sum() returned zero for every column, so no imputation was needed. The class counts are 284,315 genuine and 492 fraud. Fraud amounts are not simply bigger: the mean is 122.21 for fraud versus 88.29 for genuine, but the median is only

9.25 versus 22.00, so many fraud cases are small amounts. The genuine class also has a much larger maximum (25,691.16 versus 2,125.87). I did not capture an explicit duplicate-row check in my notebook outputs, so I make no claim about duplicates.

3. Data preprocessing

The data is already numeric and complete, so preprocessing was short. I standardised the features (zero mean, unit variance) with StandardScaler inside a scikit-learn Pipeline. The scaler was fitted only on the balanced training data and the learned transformation was then applied to the untouched test data, so no test information leaks into preprocessing. Standardising matters for KNN, SVM and Logistic Regression because they are sensitive to feature scale; the Decision Tree does not need it but is not harmed by it. The large Amount values in particular would otherwise dominate distance calculations. For the earlier Logistic Regression notebook I also dropped Time and scaled Amount separately into std_Amount. I did not remove outliers, because in fraud data the extreme values are often exactly the cases of interest.

The text steps from the brief (titles, abstracts, URL clean-up) apply to document datasets, not to this numeric table, so I left them out. The only text processing is inside the question-answering tool, where TF-IDF removes English stop words and keeps one- and two-word phrases so that terms like "class imbalance" stay together.

	Before	After
Rows	284,807	Test set unchanged; training set undersampled (Section 4)
Missing values	0	0
Features	30 raw	30 standardised

4. Feature engineering and data splitting

I used a stratified 80/20 train-test split with random_state=42, so both parts keep the original fraud rate. The test set has 56,962 transactions, of which 98 are fraud, and I left it exactly as it came. Random undersampling was then applied **only to the training data**, which gave 788 rows (394 genuine and 394 fraud). Every model was trained on those same 788 rows and scored on the same 56,962 test rows.

Set	Rows	Fraud	Treatment
Training (after undersampling)	788	394	Balanced 50/50, random undersampling
Test	56,962	98	Original distribution, never resampled
Validation	not created	-	See limitations

How leakage is avoided. In my first notebooks I undersampled the whole dataset and then split it. That makes a tidy balanced test set but it is not realistic, and it is why those early scores (for example a Decision Tree fraud recall of 0.72 on a full-size test run and KNN around 0.80 to 0.84) cannot be compared with the final table. I treat those runs as exploration only. In the final pipeline the split comes first and the test rows are never touched afterwards. One honest gap: I did not create a separate validation set, so the model settings below were not tuned on held-out data.

5. Models and results

I implemented four models. None were tuned; I used settings that are close to defaults and recorded them.

Model	Settings	Why I chose it
Decision Tree	criterion = entropy, max_depth = None, random_state = 42	Easy to read; shows a no-assumption baseline
KNN	K = 1 (distance on standardised features)	Simple, but sensitive to noisy points
Logistic Regression	max_iter = 3000, random_state = 42	Linear baseline; max_iter raised after an early convergence warning at 100
SVM	kernel = rbf, probability = True, random_state = 42	Can fit curved boundaries with few training rows

Model	Acc.	F-Prec.	F-Recall	F-F1	Macro-F1	ROC-AUC	PR-AUC	Train s	Pred. s
Decision Tree	0.9165	0.0180	0.8878	0.0353	0.4958	0.9022	0.0162	0.1763	0.0601
KNN (K=1)	0.9300	0.0219	0.9082	0.0428	0.5032	0.9191	0.0200	0.0213	0.2782
Logistic Reg.	0.9708	0.0516	0.9184	0.0977	0.5414	0.9752	0.4578	0.1513	0.0745
SVM (rbf)	0.9742	0.0563	0.8878	0.1060	0.5464	0.9733	0.5909	0.1967	3.3126

Table 1. Common evaluation on the same 56,962-row test set (F- = fraud class; Pred. = time to predict the whole test set). SVM has the best Macro-F1 and PR-AUC.

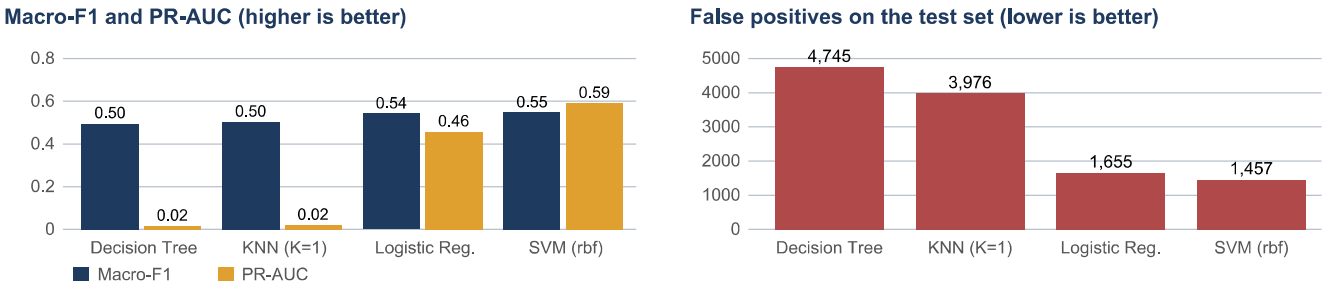


Figure 2. Macro-F1 and PR-AUC (left) and false positives (right) for each model, from Tables 1 and 2.

Confusion matrices and error analysis

Model	True neg.	False pos.	False neg.	True pos.	Frauds caught	Genuine flagged
Decision Tree	52,119	4,745	11	87	87 of 98	4,745
KNN (K=1)	52,888	3,976	9	89	89 of 98	3,976
Logistic Reg.	55,209	1,655	8	90	90 of 98	1,655
SVM (rbf)	55,407	1,457	11	87	87 of 98	1,457

Table 2. Rows are actual classes, columns are predicted classes.

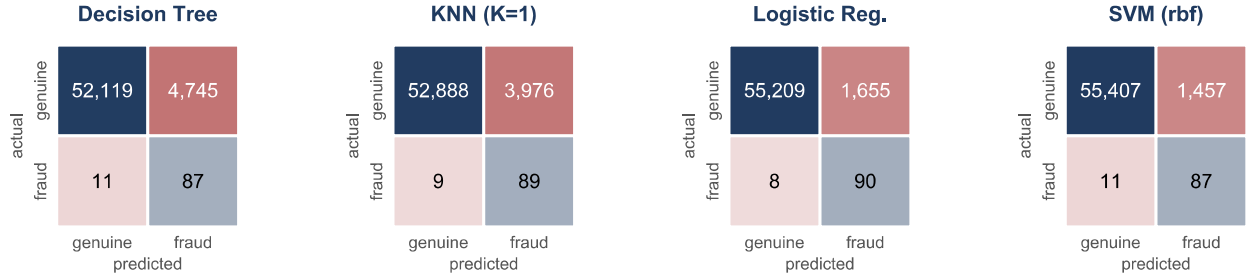


Figure 3. Confusion matrices for the four models on the same test set (darker blue = correct, darker red = errors).

Reading the numbers. All four models find most of the fraud (87 to 90 of the 98 cases), so recall is not what separates them. What separates them is how many genuine transactions they wrongly flag. The Decision Tree flags 4,745 genuine transactions to catch 87 frauds, which means roughly 55 false alarms for every real hit. SVM flags 1,457, about 17 for every hit. That is why accuracy looks fine for all of them (92% to 97%) while fraud precision stays between 2% and 6%.

Why precision is so low. The models learned from a 50/50 training set, so they expect fraud to be common. Real traffic is 0.17% fraud, so even a small false-positive rate produces a lot of false alarms compared with the 98 real cases. This is a consequence of my design choice, not a bug. In practice the decision threshold would be raised using the predicted probabilities, or the probabilities would be recalibrated to the real fraud rate.

Why the models differ. The unrestricted Decision Tree and the K = 1 KNN both memorise individual training rows, and with only 788 rows that makes them react to noise, which fits their high false-positive counts. I did not run experiments to confirm this, so I give it as the most likely explanation, not a proven one. Logistic Regression and SVM learn smoother boundaries and generalise better, which shows in their ROC-AUC (about 0.975) and PR-AUC, where SVM (0.591) is clearly ahead of Logistic Regression (0.458) and well ahead of the other two (about 0.02).

Final model: SVM (rbf). I chose SVM on evidence, not on accuracy. It has the highest macro-F1 (0.5464), the highest PR-AUC (0.5909), the highest fraud F1 (0.1060) and the fewest false positives (1,457). The cost is speed: it took 3.31 seconds to score the test set against 0.07 for Logistic Regression, which is about 58 microseconds per transaction and still acceptable for real-time use. Logistic Regression is a very close second and catches three more frauds (90 versus 87), so if recall mattered most, or if latency were tight, I would choose it. The gap between the two is small and comes from a single split and a single seed, so I would not call SVM the winner with confidence without repeating the run with cross-validation.

6. Question-answering and source grounding

The question-answering part is a small TF-IDF retrieval system. The knowledge base is 10 short text entries (dataset, fraud, features, time, amount, imbalance, preprocessing, evaluation, metrics, models). TfidfVectorizer from scikit-learn uses English stop words and unigrams plus bigrams. A question is turned into a vector, compared with every entry by cosine similarity, and the single best entry (Top-1) is returned together with its ID, so every answer points back to a source. If the best score is below 0.10 the question counts as not covered. No embedding model is

used.

Question	Retrieved entry	Similarity	Right entry?
How many transactions are in the dataset?	imbalance	0.3400	No (expected dataset)
How many fraudulent transactions are present?	imbalance	0.3865	Partly (fraud holds the count)
What does Class 1 mean?	fraud	0.4040	Yes
What are V1 to V28?	features	0.3847	Yes
What does the Time feature represent?	time	0.2133	Yes
What does Amount represent?	imbalance	0.1724	No (expected amount)
Why is the dataset imbalanced?	imbalance	0.2906	Yes
How is the dataset split for evaluation?	evaluation	0.3433	Yes
What are precision and recall?	metrics	0.4246	Yes
Which algorithms are compared?	models	0.2098	Yes

Table 3. Retrieval test (ten questions). I judged "right entry" by which entry I would expect to answer the question.

Seven of ten questions hit the expected entry, one is borderline and two missed. I did **not** calculate Recall@k or MRR, because only the top result is kept. The misses happen because the "imbalance" entry is long and shares many words with the question, while "dataset" and "amount" are short. A simple fix would be to add more wording to the short entries or to retrieve the top 3 and rerank.

7. Testing, edge cases and limitations

The brief lists ten scenarios. I turned each one into a concrete test against the retrieval tool and wrote a notebook for them (*Phase6_Edge_Case_Tests.ipynb*). It rebuilds the TF-IDF index with the Phase 4 settings, changes the knowledge base where a scenario needs it (a very short entry, a duplicate, a conflicting entry, an entry with misleading instructions), and records the retrieved entry, score, runner-up and a pass or fail against the expected behaviour below. **I have not yet recorded its outputs on my knowledge base, so I report no actual results here** and do not guess them. The "I expect" column is my prediction from how TF-IDF with Top-1 works, not a measured result.

No.	Scenario	Test	Expected behaviour	I expect
1	One entry answers it	What does Class 1 mean?	Returns the fraud entry	Pass
2	Needs several entries	Which metrics and algorithms are compared?	Uses both metrics and models	Fails: Top-1 returns one entry
3	Technical synonyms	How many scam transactions exist?	Returns fraud or imbalance entry	May fail: no word meaning, only word overlap
4	Absent topic	What is the capital of France?	Below 0.10, says not found	Pass
5	Empty question	"" and whitespace only	Asks for a question, no crash	Pass
6	Very short entry	Time entry reduced to one word	No crash; weak answer	Pass, weak answer
7	Irrelevant passages	How do I bake a chocolate cake?	Below 0.10, says not found	Pass
8	Conflicting entries	Add a 70/30 split entry next to the 80/20 one	Should flag the conflict	Fails: ties, no conflict warning
9	Duplicate entries	Copy of the fraud entry	Same answer, no crash	Pass
10	Misleading text	Entry that says to ignore instructions	Must not follow it	Risk: text is returned as-is

Table 4. Phase 6 test plan. The same notebook also calculates Recall@1, Recall@3 and MRR on the ten normal questions, which closes the metric gap noted above once it is run.

Main limitations

- No validation set, no hyperparameter tuning, and one split with one seed, so small differences between models are not reliable.
- Fraud precision is very low (2% to 6%) at the default threshold; the threshold has not been tuned.
- The dataset is a two-day, anonymised European snapshot; fraud patterns change over time.
- Earlier exploratory runs used other splits and are not comparable with Table 1.
- The retrieval prototype is Top-1 only, got 3 of 10 questions not clearly right, and its edge-case outputs are not yet recorded.

Before real deployment I would add a validation set and cross-validation, tune the decision threshold against a cost for missed fraud versus false alarms, test on later time periods, monitor drift, and keep a human reviewer in the loop for flagged cases.

8. Explaining the implementation

Data flow. creditcard.csv is loaded with pandas, checked for nulls and class balance, and split into training and test sets with stratification. The features are standardised, the training rows are undersampled to 394 genuine and 394 fraud, and each of the four models is fitted on those 788 rows. Predictions on the untouched test set are turned into a classification report, confusion matrix, ROC-AUC and PR-AUC, and the results are saved to a comparison CSV.

Why these models and metrics. I wanted four different ideas on the same data: rules (Decision Tree), nearest neighbours (KNN), a linear boundary (Logistic Regression) and a kernel boundary (SVM). Accuracy is misleading when 99.8% of rows are genuine, so I report fraud precision and recall, macro-F1 (which weights both classes equally), and PR-AUC, which is the most honest single number for a rare positive class. ROC-AUC is included because it is commonly reported, but it looks optimistic when negatives dominate.

How retrieval works and how answers are grounded. TF-IDF gives each word a weight that is high when the word is frequent in one entry but rare across the knowledge base. Cosine similarity then measures how close the question vector is to each entry vector. The answer is the text of the best entry and the entry ID is shown with it, so anyone can check where an answer came from. Grounding is only as good as the retrieval: the Amount and dataset-size misses in Table 3 show it can return a related but wrong entry.

Declaration. I, Navadeep Reddy, confirm that this work was completed under the stated assessment rules, that results are taken from my own notebook outputs, and that the dataset and its licence are acknowledged above.
Signature: **Navadeep Reddy** Date: 30 October 2026